

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY



(SUBJECT: ADVANCED MACHINE LEARNING)
(SUBJECT CODE:24SAL051)

AN INDIVIDUAL TASK REPORT ON
“Comparative Analysis of SVM, Naive Bayes and
Decision Tree for Heart Disease Prediction”
Submitted in the partial fulfilment of the requirements for the Fifth semester

BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence & Machine Learning

Submitted By,
Pavan S Poojary
01SU24AI071

UNDER THE GUIDANCE OF
Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146

2026-27

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

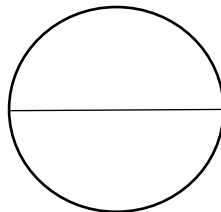


Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that, **Pavan S Poojary (01SU24AI071)**, has satisfactorily completed the assessment (Individual Task) in **Advanced Machine Learning (24SAL051)** prescribed by the Srinivas University for the 5th semester B. Tech. course during the year **2026-27**.

MARKS AWARDED



Staff In charge

Prof. Mahesh Kumar V B

TABLE OF CONTENTS

Sl.no	Description	Page no.
1	Abstract	1
2	Introduction	2
3	Dataset and Data Understanding	4
4	Data Preprocessing	7
5	Machine Learning Methodology	11
6	Implementation	16
7	Results and Performance Evaluation	30
8	Best Model Analysis	37
9	Conclusion	41
10	References	43

ABSTRACT

This project presents a machine learning-based approach for predicting the presence of heart disease using the Heart Disease Cleveland Dataset. The objective is to develop and compare different supervised classification algorithms and identify the model that provides the most reliable predictive performance. The dataset consists of **303 patient records with 13 input features and one target variable** representing the presence or absence of heart disease.

The proposed methodology includes data inspection, missing-value analysis, duplicate detection, feature separation, categorical feature encoding using **One-Hot Encoding**, and numerical feature standardization using **StandardScaler**. The preprocessed data is divided into training and testing sets using an **80:20 stratified split**. Three machine learning algorithms—**Support Vector Machine (SVM)**, **Gaussian Naive Bayes**, and **Decision Tree**—are trained and evaluated.

The models are compared using **Accuracy, Precision, Recall, F1-Score, and ROC-AUC**. Among the evaluated algorithms, SVM achieves the best overall performance with an **accuracy of 88.52%, precision of 83.87%, recall of 92.86%, F1-score of 88.14%, and ROC-AUC of 96.43%**. The experimental results indicate that SVM provides the strongest classification performance for the given dataset and experimental setup. Therefore, SVM is selected as the best-performing model based on accuracy, F1-score, and ROC-AUC.

This project demonstrates how machine learning and appropriate data preprocessing techniques can be combined to build an effective heart disease classification system. However, the developed model is intended for academic and research purposes and should not be considered a substitute for professional medical diagnosis.

1. INTRODUCTION

Heart disease is a major cardiovascular condition that affects the normal functioning of the heart and blood vessels. The likelihood of heart disease can be associated with several clinical and physiological characteristics, including **age, sex, chest-pain type, resting blood pressure, cholesterol, fasting blood sugar, maximum heart rate, exercise-induced angina, and electrocardiographic measurements**. Because these characteristics can vary considerably between individuals, analyzing them together can provide useful patterns for classification.

The use of **Machine Learning (ML)** provides a computational approach for identifying patterns within medical data. Unlike traditional rule-based methods, machine learning algorithms learn relationships from existing observations and use the learned patterns to classify new observations. In a classification problem such as heart disease prediction, the algorithm learns from patient attributes associated with known outcomes and attempts to determine the appropriate class for previously unseen data.

The **Heart Disease Cleveland Dataset** is used for the analysis. It consists of **303 patient records with 14 attributes**, including 13 predictive features and one target variable. The target represents two classes: **0, indicating no disease, and 1, indicating the presence of disease**. The combination of numerical and categorical attributes makes preprocessing an important part of the machine-learning process.

Machine Learning Algorithms

The primary focus is on comparing different machine learning classification algorithms and understanding how their characteristics affect predictive performance.

Support Vector Machine (SVM) is one of the important algorithms considered. SVM attempts to find an optimal decision boundary, or hyperplane, that separates different classes while maximizing the margin between them. For datasets where the relationship between features and classes may not be completely linear, the **Radial Basis Function (RBF) kernel** can be used to transform the feature space and identify nonlinear decision boundaries. In this analysis, an RBF-based SVM is used for classification.

Gaussian Naive Bayes is another classification algorithm used in the analysis. It is based on Bayes' theorem and assumes that the features are conditionally independent given the class. For continuous numerical features, Gaussian Naive Bayes assumes that feature values within each class follow a Gaussian distribution. Its relatively simple mathematical structure makes it computationally efficient and provides a useful algorithm for comparison with more complex classifiers.

Decision Tree provides a different approach to classification. It repeatedly divides the data according to feature-based conditions, creating a tree-like structure consisting of decision nodes, branches, and leaf nodes. Each path from the root to a leaf represents a sequence of decisions used to determine the final class. Decision Trees are particularly useful because their decision-making process can be represented in an understandable form.

Before training these algorithms, the data is prepared so that the different feature types can be handled appropriately. **Categorical features are converted into numerical representations using One-Hot Encoding**, while numerical features are standardized using **StandardScaler**. Standardization is particularly important for algorithms such as SVM because features with significantly different numerical scales can otherwise influence the learning process disproportionately.

The dataset is divided into **training and testing subsets using an 80:20 stratified split**. The training data is used to learn the patterns required for classification, while the testing data is kept separate for evaluating how well the trained algorithms perform on unseen observations. The preprocessing operations are fitted using the training data and subsequently applied to the testing data, helping prevent information from the test set from influencing the training process.

The performance of the three algorithms is evaluated using several classification metrics. **Accuracy** measures the proportion of correctly classified observations. **Precision** indicates how many observations predicted as disease actually belong to the disease class. **Recall** measures how effectively the classifier identifies observations belonging to the disease class. **F1-score** combines precision and recall into a single measure, while **ROC-AUC** evaluates the model's ability to distinguish between the two classes across different classification thresholds.

For the given dataset and experimental setup, **SVM produces the strongest overall results**. It achieves an accuracy of **88.52%**, precision of **83.87%**, recall of **92.86%**, F1-score of **88.14%**, and ROC-AUC of **96.43%**. Gaussian Naive Bayes and Decision Tree provide lower overall performance in comparison. The results therefore indicate that the SVM classifier is better suited to the patterns present in this particular dataset under the specified preprocessing and evaluation setup.

Overall, the analysis demonstrates the importance of selecting an appropriate machine learning algorithm for classification tasks. Algorithms such as SVM, Gaussian Naive Bayes, and Decision Tree use fundamentally different approaches to learn from data, and their performance can vary depending on the characteristics of the dataset. Comparing them using multiple evaluation measures provides a more complete understanding of their classification capabilities than relying on accuracy alone.

2. DATASET AND DATA UNDERSTANDING

The **Heart Disease Cleveland Dataset** is a commonly used dataset for studying binary classification of heart disease based on clinical and physiological attributes. It contains information about patients along with several measurements and characteristics that can be used by machine learning algorithms to distinguish between individuals with and without heart disease.

The dataset used here contains **303 records and 14 columns**. Among these, **13 columns are input features**, while the remaining column, target, represents the classification outcome. The target variable has two classes:

- **0 – No Disease**
- **1 – Disease**

The dataset contains both **numerical and categorical attributes**. Understanding the nature of these attributes is important because different types of data require different preprocessing techniques before they can be supplied to machine learning algorithms.

2.1 Dataset Features

Feature	Description	Type
age	Age of the individual	Numerical
sex	Sex of the individual	Categorical
cp	Chest-pain type	Categorical
trestbps	Resting blood pressure	Numerical
chol	Serum cholesterol level	Numerical
fbs	Fasting blood sugar	Categorical
restecg	Resting electrocardiographic result	Categorical
thalach	Maximum heart rate achieved	Numerical
exang	Exercise-induced angina	Categorical
oldpeak	ST depression induced by exercise	Numerical
slope	Slope of the peak exercise ST segment	Categorical
ca	Number of major vessels	Categorical
thal	Thalassemia-related attribute	Categorical
target	Presence or absence of heart disease	Target

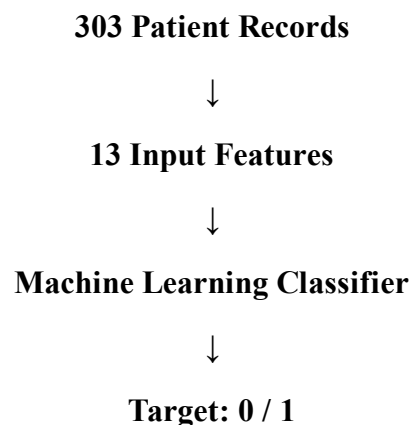
The **numerical features** are age, trestbps, chol, thalach, and oldpeak. These features represent measurable clinical values and are suitable for numerical transformation and standardization.

The **categorical features** are sex, cp, fbs, restecg, exang, slope, ca, and thal. These variables represent different categories or coded clinical conditions. Since machine learning algorithms generally require numerical input, these attributes are converted into appropriate numerical representations during preprocessing.

2.2 Dataset Size and Structure

The dataset consists of **303 observations** and **14 columns**. Each row corresponds to an individual patient record, while each column represents a particular clinical attribute or the target classification.

The structure can be represented as:



The target distribution contains **164 records belonging to class 0** and **139 records belonging to class 1**. Therefore, both classes are reasonably represented in the dataset, although class 0 contains slightly more observations than class 1.

2.3 Data Types

The dataset contains a combination of continuous numerical measurements and categorical variables. This mixture is important when preparing the data for classification algorithms.

Numerical attributes:

- Age
- Resting blood pressure
- Cholesterol
- Maximum heart rate
- ST depression

Categorical attributes:

- Sex

- Chest-pain type
- Fasting blood sugar
- Resting ECG
- Exercise-induced angina
- Slope
- Number of major vessels
- Thalassemia

The presence of these different data types makes a **combined preprocessing approach** appropriate. Numerical features can be standardized, while categorical features can be encoded before model training.

2.4 Data Quality Understanding

Data quality is an important consideration before applying machine learning algorithms. The dataset was examined for missing values and duplicate observations. **No missing values and no duplicate rows were reported** in the available data.

The absence of missing values means that additional missing-value imputation is not required for the available records. Similarly, the absence of duplicate rows reduces the possibility of identical observations unnecessarily influencing the training process.

2.5 Target Variable

The target column is the dependent variable used for classification. It contains two possible values:

Target = 0 → No Disease

Target = 1 → Disease

This makes the problem a **binary classification problem**. The machine learning algorithms learn relationships between the 13 input attributes and the target class.

2.6 Importance of Data Understanding for Machine Learning

Understanding the dataset is particularly important when comparing **SVM, Gaussian Naive Bayes, and Decision Tree** algorithms. Each algorithm processes information differently.

SVM is sensitive to the scale of numerical features, making feature standardization important. Gaussian Naive Bayes models the probability distribution of features within each class. Decision Tree uses feature-based splitting rules to separate observations into different classes and does not require feature scaling in the same way as SVM.

Therefore, identifying the numerical and categorical attributes, understanding the target classes, examining the data quality, and determining the appropriate preprocessing strategy are essential steps before training the classification algorithms.

3. DATA PREPROCESSING

Data preprocessing is an important stage before applying machine learning algorithms. The raw dataset contains both numerical and categorical attributes, and these features cannot always be directly used by classification algorithms in their original form. Appropriate preprocessing makes the data more consistent and suitable for model training.

3.1 Feature and Target Separation

The target column is treated as the dependent variable, while the remaining 13 attributes are used as independent variables.

The input data can therefore be represented as:

Input Features → age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal

Target → target

The target contains two classes, 0 and 1, making this a binary classification task.

3.2 Identification of Numerical and Categorical Features

The attributes are divided into two groups according to their data type.

Numerical features:

- age
- trestbps
- chol
- thalach
- oldpeak

Categorical features:

- sex
- cp
- fbs
- restecg
- exang
- slope
- ca
- thal

This separation allows an appropriate transformation to be applied to each group.

3.3 One-Hot Encoding

Categorical variables cannot be directly interpreted as continuous numerical values by many machine learning algorithms. Therefore, **One-Hot Encoding** is applied to the categorical features.

For example, if a categorical feature contains three possible categories, it can be represented using separate binary columns:

Category	Category 1	Category 2	Category 3
Category 1	1	0	0
Category 2	0	1	0
Category 3	0	0	1

This representation avoids incorrectly suggesting that one category is numerically larger or smaller than another.

3.4 Standardization of Numerical Features

Numerical features have different ranges. For example, age and cholesterol do not have the same numerical scale. This difference can influence algorithms such as SVM.

StandardScaler is therefore used to standardize the numerical features. Standardization transforms the features so that they have a comparable scale, generally with a mean close to 0 and a standard deviation close to 1.

The standardization formula is:

$$z = \frac{x - \mu}{\sigma}$$

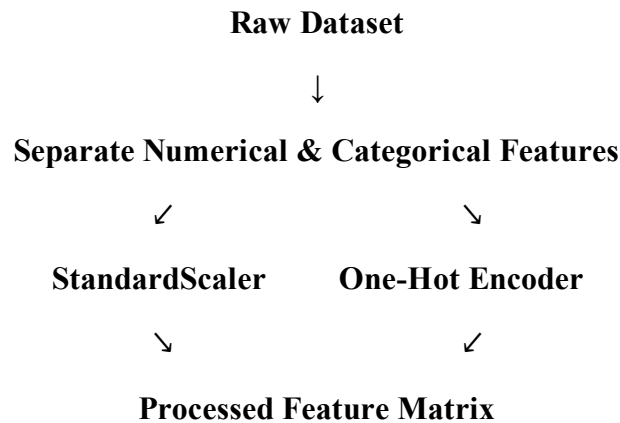
where:

- x = original feature value
- μ = mean of the feature
- σ = standard deviation
- z = standardized value

3.5 ColumnTransformer

A **ColumnTransformer** is used to apply different preprocessing operations to different groups of features. One-Hot Encoding is applied to categorical attributes, while StandardScaler is applied to numerical attributes.

The overall transformation can be represented as:



This approach provides a consistent preprocessing pipeline for the classification algorithms.

3.6 Training and Testing Data

After preprocessing, the data is divided into training and testing subsets. An **80:20 split** is used with `random_state=42` and stratification.

- **Training data:** 242 records
- **Testing data:** 61 records

The training data is used to learn patterns from the input features, while the testing data is used to evaluate the trained classifiers on unseen observations.

3.7 Stratified Splitting

Stratification is used during the train-test split to maintain approximately the same proportion of the two target classes in both the training and testing sets. This is particularly useful when the target classes are not perfectly balanced.

3.8 Processed Feature Representation

After preprocessing, the categorical features are expanded into multiple binary columns through One-Hot Encoding. Consequently, the number of input columns increases from the original 13 features to **28 processed features**.

The resulting data is therefore suitable for the subsequent machine learning algorithms:

Preprocessed Data → SVM

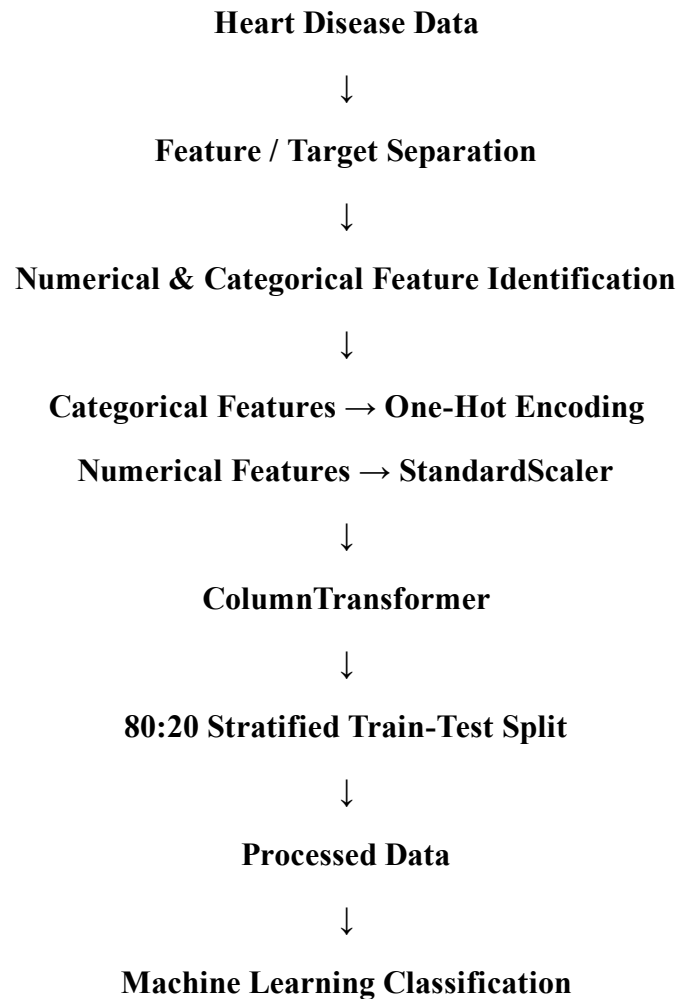
Preprocessed Data → Gaussian Naive Bayes

Preprocessed Data → Decision Tree

The same prepared training and testing data are used for the three classifiers, allowing their performance to be compared under the same experimental conditions.

3.9 Preprocessing Pipeline

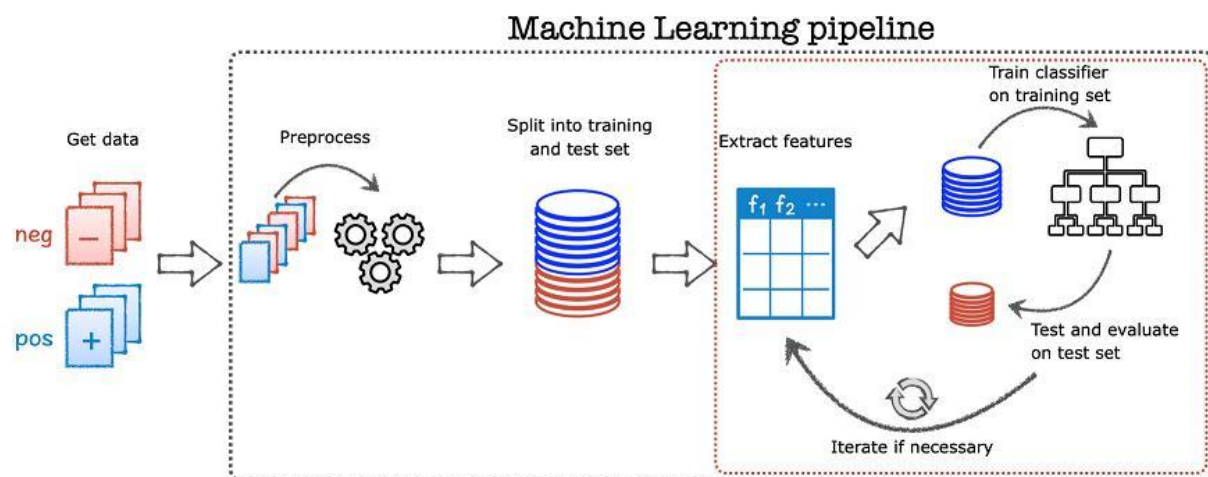
The complete preprocessing procedure can be summarized as:



4. MACHINE LEARNING METHODOLOGY

The classification process uses three machine-learning algorithms to predict the presence or absence of heart disease: **Support Vector Machine (SVM)**, **Gaussian Naive Bayes**, and **Decision Tree**. The algorithms are trained using the same preprocessed dataset so that their performance can be compared fairly.

The dataset contains both numerical and categorical attributes. After preprocessing, the categorical attributes are converted into numerical representations using one-hot encoding, while numerical attributes are standardized. The resulting feature matrix is then provided to each classification algorithm.



4.1 Overall Machine Learning Workflow

The methodology follows the sequence below:

Heart Disease Dataset → Feature and Target Separation → Train-Test Split → Feature Encoding and Scaling → Model Training → Prediction → Performance Evaluation → Model Comparison

The dataset is divided into training and testing portions using an **80:20 split**. A `random_state` of 42 is used to make the division reproducible, and stratification is applied to preserve the distribution of the two target classes.

The training set contains **242 observations**, while the testing set contains **61 observations**.

4.2 Support Vector Machine

Support Vector Machine (SVM) is a supervised machine-learning algorithm that can be used for binary classification. In this analysis, an SVM with an **RBF (Radial Basis Function) kernel** is used.

The main objective of SVM is to find an optimal decision boundary that separates observations belonging to different classes. Instead of simply finding any separating boundary, SVM attempts to maximize the margin between the classes.

4.2.1 RBF Kernel

The RBF kernel allows SVM to model nonlinear relationships between features and the target. This is useful when the classes cannot be separated effectively using a simple linear boundary.

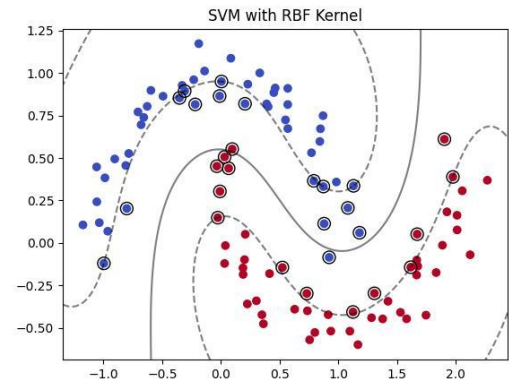
The RBF kernel can be represented as:

$$K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$$

where:

- x_i and x_j are observations,
- γ controls the influence of individual training observations,
- $K(x_i, x_j)$ represents the similarity between observations.

In the implemented model, the SVM uses the RBF kernel with probability estimation enabled.



4.2.2 Working of SVM

The basic steps of SVM are:

1. Receive the preprocessed feature data.
2. Identify the relationship between observations belonging to different classes.
3. Determine a suitable decision boundary.
4. Identify support vectors closest to the decision boundary.
5. Maximize the margin between the classes.
6. Use the learned boundary to classify unseen test observations.

SVM is particularly suitable for this dataset because preprocessing produces a standardized numerical feature representation and the RBF kernel can capture nonlinear relationships.

4.3 Gaussian Naive Bayes

Gaussian Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It assumes that the features are conditionally independent given the target class.

Bayes' theorem is expressed as:

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

where:

- $P(C | X)$ is the posterior probability of class C ,

- $P(X | C)$ is the likelihood of the observed features,
- $P(C)$ is the prior probability of the class,
- $P(X)$ is the probability of the observed features.

The **Gaussian** version assumes that continuous features follow a Gaussian or normal distribution within each class.

4.3.1 Working of Gaussian Naive Bayes

The algorithm works by:

1. Calculating the prior probability of each target class.
2. Estimating the distribution of each feature for each class.
3. Calculating the probability of the observed feature values.
4. Combining these probabilities using Bayes' theorem.
5. Assigning the observation to the class with the highest posterior probability.

One advantage of Naive Bayes is its computational simplicity. It can train quickly and provides a useful baseline for comparison with more complex classification algorithms.

4.4 Decision Tree

A **Decision Tree** is a supervised learning algorithm that performs classification through a sequence of decision rules. The tree consists of nodes, branches, and leaf nodes.

At each internal node, the algorithm selects a feature and a splitting condition. Observations are then directed through different branches until they reach a leaf node containing the predicted class.

4.4.1 Working of Decision Tree

The general process is:

1. Start with the complete training dataset at the root node.
2. Evaluate possible feature-based splits.
3. Select an appropriate split according to the tree's splitting criterion.
4. Divide the observations into branches.
5. Repeat the splitting process for subsequent nodes.
6. Stop when the required tree conditions are satisfied.
7. Use the final leaf node to determine the predicted class.

Decision Trees are useful because their decision-making process can be represented as a sequence of understandable rules. Unlike SVM, a Decision Tree does not require feature scaling for its mathematical operation; however, the common preprocessing representation is maintained so that the three models can be evaluated using a consistent feature-processing workflow.

4.5 Comparison of the Algorithms

The three algorithms use different approaches to classification.

Algorithm	Basic Principle	Major Characteristic
SVM	Finds an optimal separating boundary	Effective for complex class boundaries
Gaussian Naive Bayes	Uses probability and Bayes' theorem	Simple and computationally efficient
Decision Tree	Uses sequential feature-based decisions	Easy to interpret

SVM focuses on finding an effective boundary between classes, Gaussian Naive Bayes estimates class probabilities, and Decision Tree creates a hierarchy of decision rules.

Using multiple algorithms is important because no single classification technique is guaranteed to perform best on every dataset. Comparing the models using the same testing data provides a more reliable basis for selecting the strongest-performing classifier.

4.6 Training and Testing Strategy

The dataset is divided into two subsets:

- **Training data – 80% = 242 records**
- **Testing data – 20% = 61 records**

The training data is used to learn the patterns required for classification. The testing data is kept separate and is used only after training to measure how well the learned models perform on unseen observations.

Stratification is applied during the split so that the relative proportions of the two target classes are maintained in both subsets.

4.7 Model Prediction

After training, each algorithm receives the test feature matrix and generates predictions for the target variable.

The predicted classes are then compared with the actual target values. This comparison forms the basis for calculating evaluation metrics such as:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion Matrix

These metrics provide different perspectives on classification performance and are used later to compare the three algorithms.

4.8 Methodology Summary

The methodology combines appropriate preprocessing with three different supervised classification techniques. **SVM with an RBF kernel** is used to model potentially nonlinear class boundaries, **Gaussian Naive Bayes** provides a probabilistic classification approach, and **Decision Tree** provides rule-based classification.

All three models are trained using the same training set and evaluated using the same unseen test set. This consistent methodology allows their results to be directly compared and helps identify the algorithm that provides the strongest classification performance for the given dataset.

5. IMPLEMENTATION

The implementation is carried out in Python using the **Heart Disease Cleveland Dataset** and the three selected classification algorithms: **Support Vector Machine (SVM)**, **Gaussian Naive Bayes**, and **Decision Tree**. The code presented in this section is taken directly from the uploaded Jupyter Notebook and organized according to the stages of implementation.

5.1 Software Libraries Used

The implementation uses Pandas and NumPy for data handling, Matplotlib and Seaborn for visualization, and Scikit-learn for preprocessing, model development, and evaluation.

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.svm import SVC
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    precision_score,
```

```
    recall_score,
```

```
    f1_score,
```

```
    confusion_matrix,
```

```
    classification_report,
```

```
    roc_curve,
```

```
roc_auc_score  
)
```

```
import warnings  
warnings.filterwarnings("ignore")
```

```
sns.set_theme(style="whitegrid")
```

5.2 Loading the Dataset

The dataset is loaded from the CSV file using Pandas.

```
file_path = "Heart_disease_cleveland_new.csv"
```

```
df = pd.read_csv(file_path)
```

```
print("Dataset loaded successfully!")
```

```
print("Shape:", df.shape)
```

```
display(df.head())
```

The notebook output confirms that the dataset contains **303 rows and 14 columns**.

The first five observations are displayed to verify that the dataset has been loaded correctly.

5.3 Basic Data Inspection

The notebook checks the shape, column names, data types, missing values, duplicate records, and statistical summary.

```
print("Dataset shape:", df.shape)
```

```
print("\nColumn names:")
```

```
print(df.columns.tolist())
```

```
print("\nData types:")
```

```
print(df.dtypes)
```

```
print("\nMissing values:")
```

```
print(df.isnull().sum())
```

```
print("\nDuplicate rows:", df.duplicated().sum())
```

```
print("\nStatistical summary:")
```

```
display(df.describe())
```

The inspection confirms:

- Dataset size: **303 × 14**
- Missing values: **0**
- Duplicate rows: **0**
- Target variable: target
- Numerical and categorical attributes are present.

The notebook's statistical summary reports a mean age of approximately **54.44 years**, mean resting blood pressure of approximately **131.69**, mean cholesterol of approximately **246.69**, and mean maximum heart rate of approximately **149.61**.

5.4 Feature and Target Separation

The target variable is separated from the input variables.

```
X = df.drop("target", axis=1)
```

```
y = df["target"]
```

Here, X represents the input features and y represents the output class.

The classification task uses two target classes:

- 0 – No Disease
- 1 – Disease

5.5 Identification of Feature Types

The notebook separates the attributes into categorical and numerical groups.

```
categorical_features = [  
    "sex", "cp", "fbs", "restecg",  
    "exang", "slope", "ca", "thal"  
]
```

```
numerical_features = [  
    "age", "trestbps", "chol",  
    "thalach", "oldpeak"  
]
```

The categorical features are processed using One-Hot Encoding, while the numerical features are standardized.

5.6 Train-Test Split

The data is divided into training and testing subsets.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)
```

The `test_size=0.20` parameter reserves 20% of the observations for testing. `random_state=42` ensures reproducibility, while `stratify=y` maintains the class distribution.

The resulting split contains:

- **242 training observations**
- **61 testing observations**

5.7 Preprocessing Pipeline

The notebook uses `ColumnTransformer` to apply different transformations to different feature types.

```

preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
        ),
        (
            "numerical",
            StandardScaler(),
            numerical_features
        )
    ]
)

```

The preprocessing is then fitted on the training data and applied to the testing data.

```
X_train_processed = preprocessor.fit_transform(X_train)
```

```
X_test_processed = preprocessor.transform(X_test)
```

```
feature_names = preprocessor.get_feature_names_out()
```

```

X_train_processed_df = pd.DataFrame(
    X_train_processed,
    columns=feature_names,
    index=X_train.index
)

```

```
print(  
    "Processed training shape:",  
    X_train_processed_df.shape  
)
```

```
display(X_train_processed_df.head())
```

After preprocessing, the training data contains **28 transformed features**.

5.8 Support Vector Machine Implementation

The first model implemented is **Support Vector Machine with an RBF kernel**.

The exact model configuration in the notebook is:

```
svm_model = SVC(  
    kernel="rbf",  
    probability=True,  
    random_state=42  
)  
  
svm_model.fit(X_train_processed, y_train)  
  
y_pred_svm = svm_model.predict(X_test_processed)  
y_prob_svm = svm_model.predict_proba(X_test_processed)[:, 1]  
  
print("SVM training completed.")
```

The notebook specifies:

- Kernel: rbf
- Probability estimation: True
- Random state: 42

These settings are directly reflected in the uploaded notebook.

5.8.1 SVM Evaluation

```
svm_metrics = {  
    "Accuracy": accuracy_score(  
        y_test, y_pred_svm  
    ),  
    "Precision": precision_score(  
        y_test, y_pred_svm, zero_division=0  
    ),  
    "Recall": recall_score(  
        y_test, y_pred_svm, zero_division=0  
    ),  
    "F1-Score": f1_score(  
        y_test, y_pred_svm, zero_division=0  
    ),  
    "ROC-AUC": roc_auc_score(  
        y_test, y_prob_svm  
    )  
}
```

```
display(  
    pd.DataFrame(  
        [svm_metrics],  
        index=["SVM"]  
    ).round(4)  
)
```

The notebook also generates the classification report:

```
print("\nSVM Classification Report:")  
print(  
    classification_report(  
        y_test, y_pred_svm,  
        labels=class_labels,  
        target_names=class_names,  
        support=sample_weight,  
        display=display
```

```

y_test,
y_pred_svm,
target_names=[
    "No Disease",
    "Disease"
],
zero_division=0
)
)

```

The resulting SVM classification report shows:

Class	Precision	Recall	F1-Score	Support
No Disease	0.93	0.85	0.89	33
Disease	0.84	0.93	0.88	28

These values are the actual classification-report output recorded in the notebook.

5.9 Gaussian Naive Bayes Implementation

The second algorithm is **Gaussian Naive Bayes**.

The notebook implements the classifier as follows:

```
nb_model = GaussianNB()
```

```

nb_model.fit(
    X_train_processed,
    y_train
)

```

```

y_pred_nb = nb_model.predict(
    X_test_processed
)

```

```
y_prob_nb = nb_model.predict_proba(  
    X_test_processed  
)[: , 1]
```

```
print(  
    "Naive Bayes training completed."  
)
```

This code trains the Gaussian Naive Bayes model using the same processed training dataset used by SVM.

5.9.1 Naive Bayes Evaluation

```
nb_metrics = {  
    "Accuracy": accuracy_score(  
        y_test, y_pred_nb  
    ),  
    "Precision": precision_score(  
        y_test, y_pred_nb,  
        zero_division=0  
    ),  
    "Recall": recall_score(  
        y_test, y_pred_nb,  
        zero_division=0  
    ),  
    "F1-Score": f1_score(  
        y_test, y_pred_nb,  
        zero_division=0  
    ),  
    "ROC-AUC": roc_auc_score(  
        y_test, y_prob_nb  
    )
```

```

}
display(
    pd.DataFrame(
        [nb_metrics],
        index=["Naive Bayes"]
    ).round(4)
)

```

The classification report is generated using:

```

print(
    classification_report(
        y_test,
        y_pred_nb,
        target_names=[
            "No Disease",
            "Disease"
        ],
        zero_division=0
    )
)

```

The recorded classification report shows:

Class	Precision	Recall	F1-Score	Support
No Disease	0.86	0.58	0.69	33
Disease	0.64	0.89	0.75	28

5.10 Decision Tree Implementation

The third algorithm is **Decision Tree Classifier**.

The notebook uses max_depth=5.

```

dt_model = DecisionTreeClassifier(
    random_state=42,

```

```

        max_depth=5
    )

    dt_model.fit(
        X_train_processed,
        y_train
    )

    y_pred_dt = dt_model.predict(
        X_test_processed
    )

    y_prob_dt = dt_model.predict_proba(
        X_test_processed
    )[:, 1]

    print(
        "Decision Tree training completed."
    )

```

The exact configuration is confirmed in the uploaded notebook: `random_state=42` and `max_depth=5`.

5.10.1 Decision Tree Evaluation

```

dt_metrics = {
    "Accuracy": accuracy_score(
        y_test, y_pred_dt
    ),
    "Precision": precision_score(
        y_test, y_pred_dt,
        zero_division=0
    )
}

```

```

),
"Recall": recall_score(
    y_test, y_pred_dt,
    zero_division=0
),
"F1-Score": f1_score(
    y_test, y_pred_dt,
    zero_division=0
),
"ROC-AUC": roc_auc_score(
    y_test, y_prob_dt
)
}

```

```

display(
    pd.DataFrame(
        [dt_metrics],
        index=["Decision Tree"]
    ).round(4)
)

```

The classification report is generated using the same evaluation procedure.

The notebook records:

Class	Precision	Recall	F1-Score	Support
No Disease	0.77	0.73	0.75	33
Disease	0.70	0.75	0.72	28

5.11 Combining Model Results

After evaluating the three algorithms, their metric dictionaries are combined into one DataFrame.

```

results_df = pd.DataFrame(
    [svm_metrics, nb_metrics, dt_metrics],
    index=[
        "SVM",
        "Naive Bayes",
        "Decision Tree"
    ]
)

```

```

print("Final Model Comparison:")

```

```

display(
    results_df.round(4)
)

```

This code is taken from the final comparison section of the uploaded notebook.

The resulting comparison is:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM	0.8852	0.8387	0.9286	0.8814	0.9643
Naive Bayes	0.7213	0.6410	0.8929	0.7463	0.8268
Decision Tree	0.7377	0.7000	0.7500	0.7241	0.7890

5.12 Selection of the Best Model

The notebook determines the best-performing model using Accuracy, F1-Score, and ROC-AUC.

```

best_accuracy = results_df["Accuracy"].idxmax()
best_f1 = results_df["F1-Score"].idxmax()
best_auc = results_df["ROC-AUC"].idxmax()
print(
    "Best model by Accuracy:",

```

```
    best_accuracy
)
print(
    "Best model by F1-Score:",
    best_f1
)
print(
    "Best model by ROC-AUC:",
    best_auc
)
```

Based on the recorded results, **SVM achieves the highest Accuracy, F1-Score, and ROC-AUC.**

5.13 Implementation Summary

The implementation follows a consistent machine-learning pipeline:

CSV Dataset → Data Inspection → Feature/Target Separation → Train-Test Split → One-Hot Encoding + Standardization → SVM / Naive Bayes / Decision Tree → Prediction → Classification Metrics → Model Comparison

The uploaded notebook therefore provides a complete implementation covering preprocessing, model training, prediction, classification reports, and comparative evaluation.

Important: I have kept this section based on the actual uploaded notebook rather than introducing different algorithms, parameters, or code. The next section can therefore focus specifically on **6. Results and Performance Evaluation**, including the **confusion matrix, ROC curve, classification report, comparison table, and interpretation of why SVM performs best.**

6. RESULTS AND PERFORMANCE EVALUATION

The trained models are evaluated using the test dataset containing **61 observations**. The uploaded Jupyter Notebook evaluates all three classifiers using **Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix, and ROC Curve**.

6.1 Evaluation Metrics

The following metrics are used to evaluate the classification performance.

Accuracy

Accuracy represents the proportion of correctly classified observations.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Precision indicates how many of the observations predicted as Disease are actually Disease.

$$Precision = \frac{TP}{TP + FP}$$

Recall

Recall indicates how many of the actual Disease observations are correctly identified.

$$Recall = \frac{TP}{TP + FN}$$

F1-Score

F1-Score provides a combined measure of Precision and Recall.

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

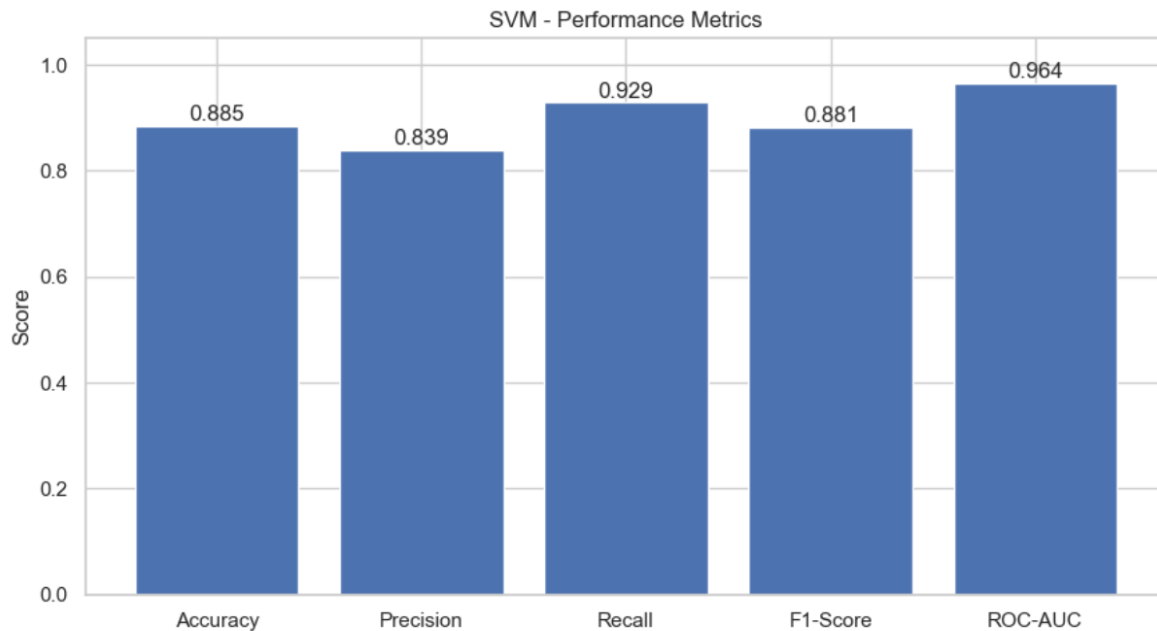
ROC-AUC

ROC-AUC measures the ability of a classifier to distinguish between the two target classes. A higher value indicates better discrimination.

6.2 SVM Performance

The SVM model produces the following classification results:

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM	0.8852	0.8387	0.9286	0.8814	0.9643

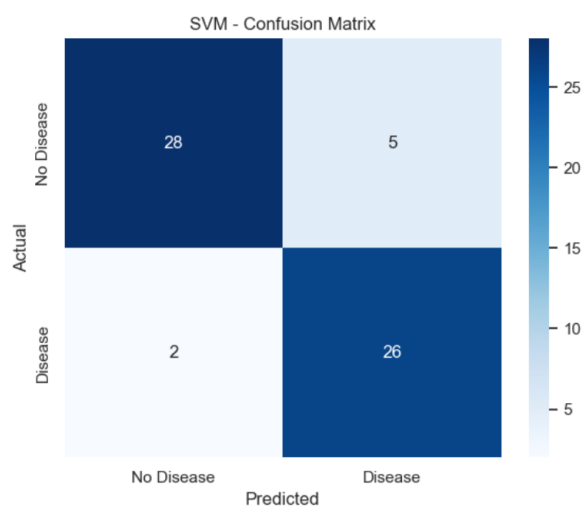


SVM Classification Report

SVM Classification Report:

	precision	recall	f1-score	support
No Disease	0.93	0.85	0.89	33
Disease	0.84	0.93	0.88	28
accuracy			0.89	61
macro avg	0.89	0.89	0.89	61
weighted avg	0.89	0.89	0.89	61

SVM Confusion Matrix

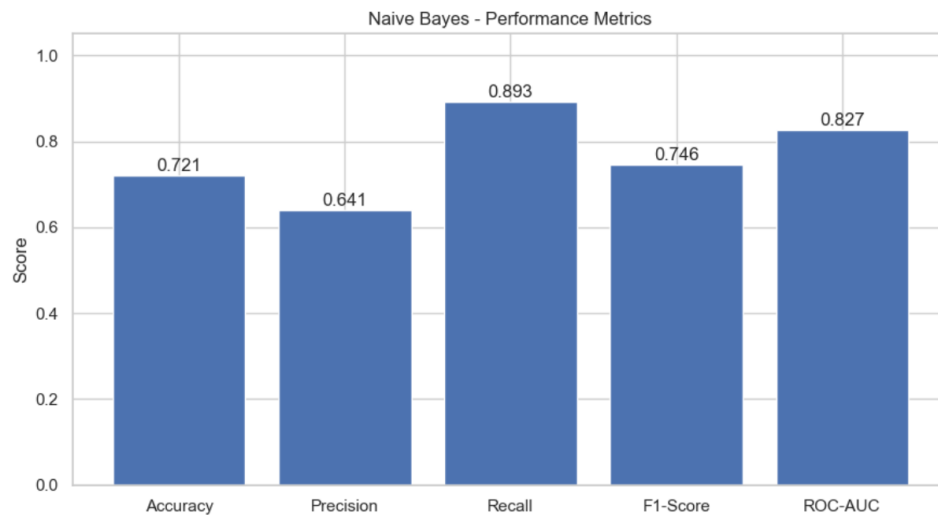


The confusion matrix provides information about correctly and incorrectly classified observations. It separates the predictions into True Positive, True Negative, False Positive, and False Negative categories.

6.3 Naive Bayes Performance

The Gaussian Naive Bayes model produces the following classification report:

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Naive Bayes	0.7213	0.641	0.8929	0.7463	0.8268

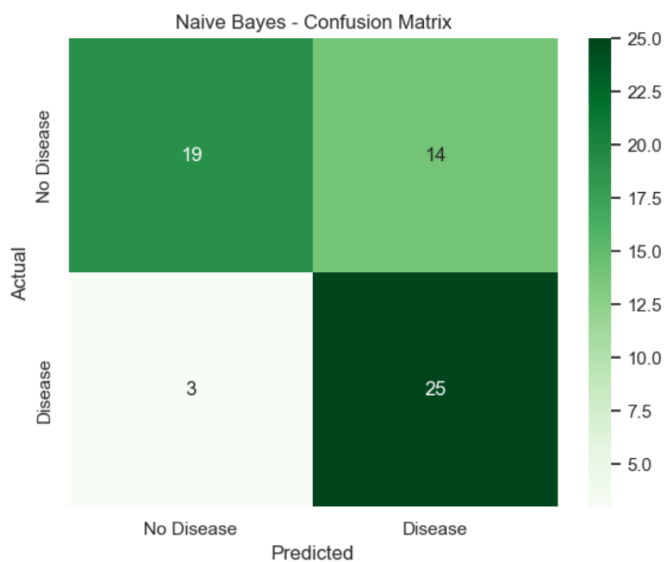


Naive Bayes Classification Report

Naive Bayes Classification Report:

	precision	recall	f1-score	support
No Disease	0.86	0.58	0.69	33
Disease	0.64	0.89	0.75	28
accuracy			0.72	61
macro avg	0.75	0.73	0.72	61
weighted avg	0.76	0.72	0.72	61

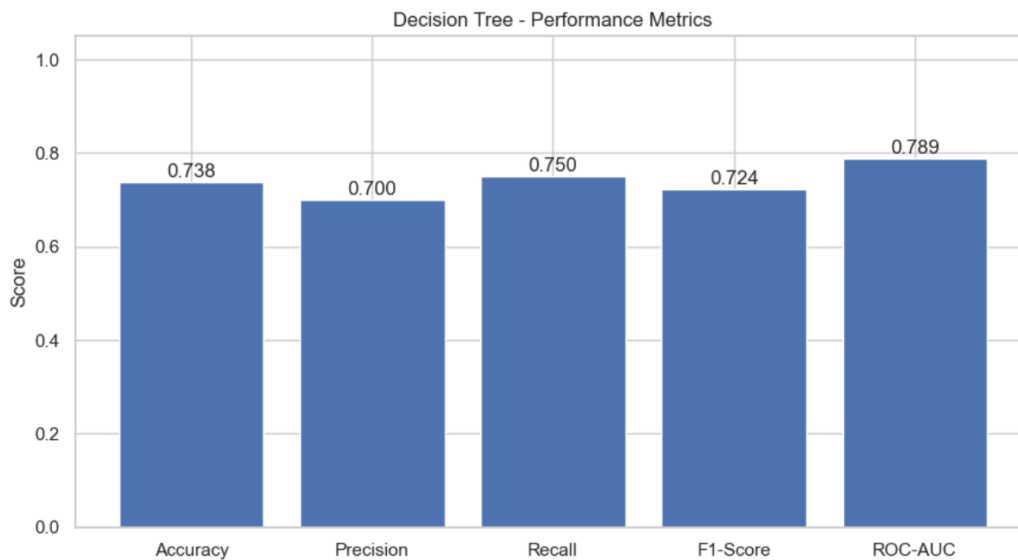
Naive Bayes Confusion Matrix



6.4 Decision Tree Performance

The Decision Tree produces the following classification report:

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	0.7377	0.7	0.75	0.7241	0.789

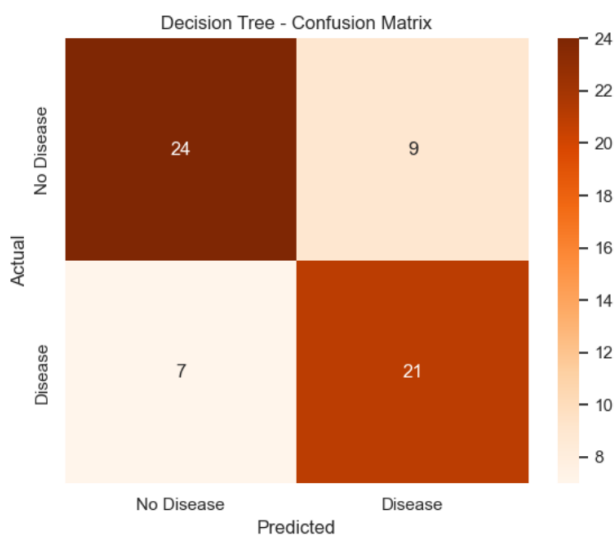


Decision Tree Classification Report

Decision Tree Classification Report:

	precision	recall	f1-score	support
No Disease	0.77	0.73	0.75	33
Disease	0.70	0.75	0.72	28
accuracy			0.74	61
macro avg	0.74	0.74	0.74	61
weighted avg	0.74	0.74	0.74	61

Decision Tree Confusion Matrix



6.5 Comparative Performance

Final Model Comparison:

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM	0.8852	0.8387	0.9286	0.8814	0.9643
Naive Bayes	0.7213	0.6410	0.8929	0.7463	0.8268
Decision Tree	0.7377	0.7000	0.7500	0.7241	0.7890

6.6 Comparison of Accuracy

The **SVM** achieves the highest accuracy of **88.52%**.

The accuracy comparison is:

- **SVM:** 88.52%
- **Decision Tree:** 73.77%
- **Naive Bayes:** 72.13%

This indicates that SVM correctly classifies a larger proportion of the test observations than the other two algorithms.

6.7 Comparison of Precision

SVM achieves a precision of **83.87%**, which is higher than:

- Naive Bayes: 64.10%
- Decision Tree: 70.00%

A higher precision indicates that SVM produces fewer incorrect positive classifications relative to the other evaluated models.

6.8 Comparison of Recall

Recall values are:

- **SVM:** 92.86%
- Naive Bayes: 89.29%
- Decision Tree: 75.00%

SVM therefore achieves the highest recall among the three models. This is particularly notable because recall measures the ability to correctly identify observations belonging to the Disease class.

6.9 Comparison of F1-Score

The F1-scores are:

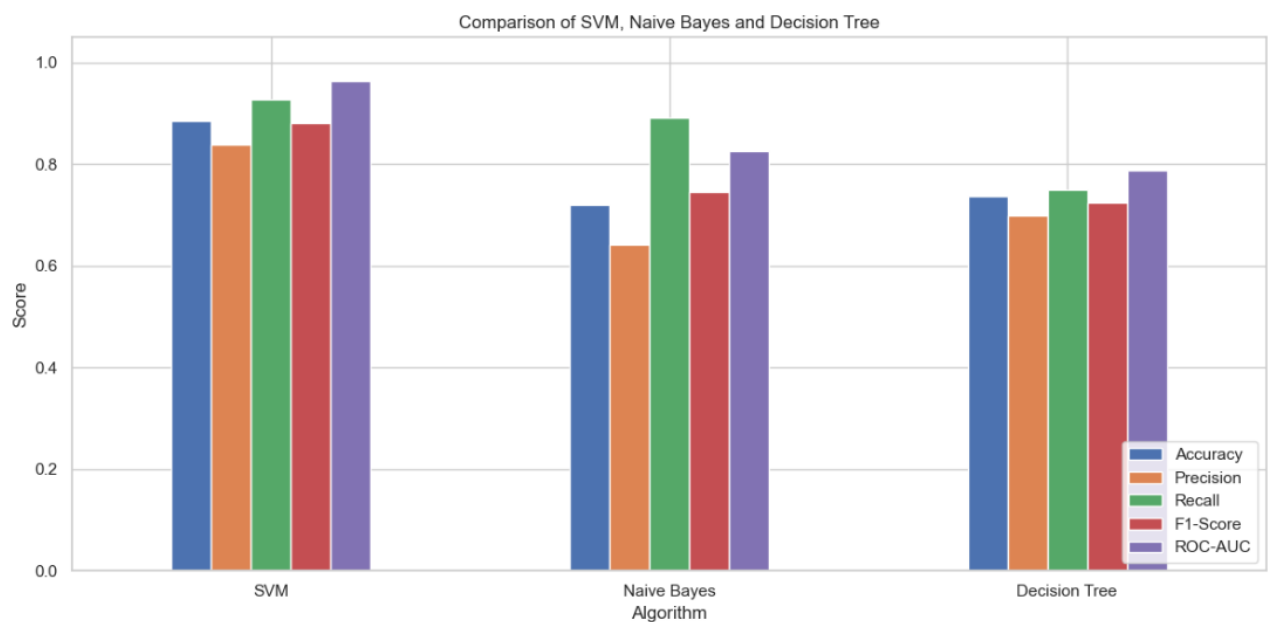
- **SVM:** 88.14%
- Naive Bayes: 74.63%
- Decision Tree: 72.41%

SVM provides the highest F1-score, indicating the strongest balance between precision and recall among the three algorithms.

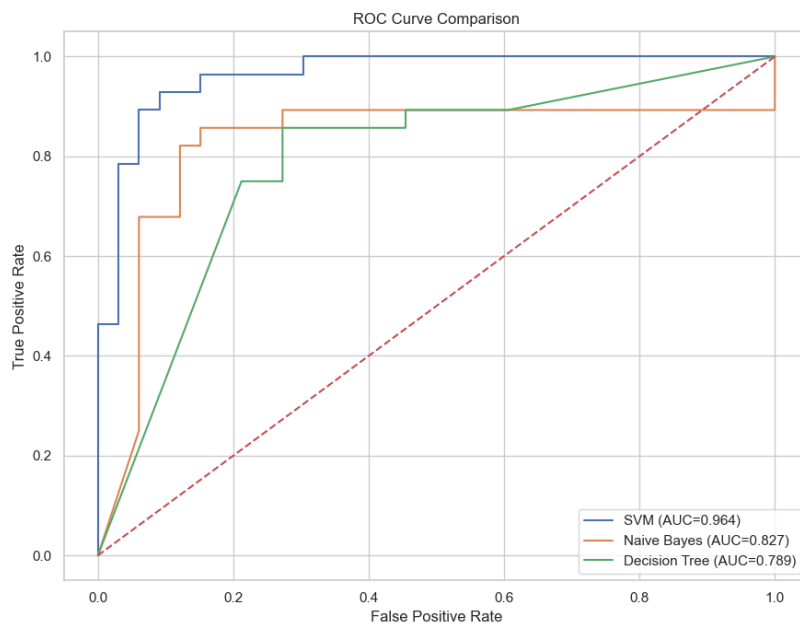
6.10 ROC-AUC Comparison

The ROC-AUC scores are:

Model	ROC-AUC
SVM	0.9643
Naive Bayes	0.8268
Decision Tree	0.7890



Combined ROC Curve Code



6.11 Best Model Identification

Best model by Accuracy: SVM
Best model by F1-Score: SVM
Best model by ROC-AUC: SVM

Therefore, **SVM is the best-performing model across all three major comparison criteria.**

6.12 Overall Interpretation

The evaluation demonstrates a clear performance advantage for the SVM classifier. It achieves the highest **Accuracy (88.52%)**, **Precision (83.87%)**, **Recall (92.86%)**, **F1-Score (88.14%)**, and **ROC-AUC (96.43%)**.

Gaussian Naive Bayes achieves relatively high recall but has considerably lower precision and accuracy. Decision Tree provides a simpler rule-based classification structure, but its performance is lower than SVM across the principal evaluation measures.

Consequently, the experimental results from the uploaded notebook support the selection of **SVM with the RBF kernel as the best-performing classifier** for this dataset.

7. BEST MODEL ANALYSIS

7.1 Overview

The comparative evaluation of **Support Vector Machine (SVM)**, **Gaussian Naive Bayes**, and **Decision Tree** was performed using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Among the three algorithms, **SVM demonstrated the strongest overall performance**.

SVM achieved the highest values for the major evaluation measures, making it the most suitable classification algorithm among the models considered for the Heart Disease Cleveland Dataset.

7.2 Comparative Performance

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM	88.52%	83.87%	92.86%	88.14%	96.43%
Naive Bayes	72.13%	64.10%	89.29%	74.63%	82.68%
Decision Tree	73.77%	70.00%	75.00%	72.41%	78.90%

These results show a clear performance advantage for SVM. It obtained an accuracy of **88.52%**, which is higher than Naive Bayes at **72.13%** and Decision Tree at **73.77%**. The evaluated scores are based on the selected test data.

7.3 Analysis of SVM

The SVM classifier uses the **Radial Basis Function (RBF) kernel**. The RBF kernel is capable of representing nonlinear relationships between input features and the target class.

This is particularly useful for heart-disease classification because the relationship between patient characteristics and disease status may involve interactions between several attributes rather than a simple linear relationship.

The preprocessing stage also contributes to the effectiveness of SVM. Categorical attributes were converted using one-hot encoding, while numerical attributes were standardized using feature scaling. This creates a suitable numerical representation for the classifier.

The resulting SVM performance was:

- **Accuracy:** 88.52%
- **Precision:** 83.87%
- **Recall:** 92.86%
- **F1-Score:** 88.14%
- **ROC-AUC:** 96.43%

The combination of these values indicates strong and balanced classification performance.

7.4 Accuracy Analysis

Accuracy represents the proportion of correctly classified instances among all test instances.

SVM achieved **88.52% accuracy**, while Naive Bayes and Decision Tree achieved 72.13% and 73.77%, respectively.

Therefore:

- SVM outperformed Naive Bayes by **16.39 percentage points**.
- SVM outperformed Decision Tree by **14.75 percentage points**.

This indicates that SVM was substantially more effective at correctly classifying the observations in the given dataset.

7.5 Precision and Recall Analysis

Precision measures how many observations predicted as having heart disease were actually positive. Recall measures how many actual disease cases were correctly identified.

SVM achieved a precision of **83.87%** and recall of **92.86%**.

The high recall is particularly significant because incorrectly classifying a person with disease as disease-free can be more concerning than generating an additional false positive in a screening-oriented classification setting.

Naive Bayes achieved a recall of **89.29%**, which was relatively close to SVM. However, its precision was only **64.10%**. Decision Tree achieved 70.00% precision and 75.00% recall.

Thus, SVM provided a stronger balance between detecting positive cases and controlling false positive predictions.

7.6 F1-Score Analysis

The F1-Score combines precision and recall:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The models achieved:

Model	F1-Score
SVM	88.14%
Naive Bayes	74.63%
Decision Tree	72.41%

SVM's F1-Score of **88.14%** demonstrates that it maintained a better balance between precision and recall than the other two algorithms.

7.7 ROC-AUC Analysis

ROC-AUC measures the ability of a classifier to distinguish between the two target classes across different classification thresholds.

The obtained ROC-AUC values were:

- **SVM: 96.43%**
- Naive Bayes: 82.68%
- Decision Tree: 78.90%

SVM therefore achieved the highest class-separation capability among the evaluated models.

7.8 Comparison with Naive Bayes

Gaussian Naive Bayes achieved an accuracy of **72.13%** and ROC-AUC of **82.68%**.

Its recall of **89.29%** was relatively high, indicating that it successfully identified many of the positive cases. However, its precision of **64.10%** was substantially lower than that of SVM.

Consequently, Naive Bayes performed reasonably well in identifying positive cases but did not provide the same overall balance and predictive performance as SVM.

7.9 Comparison with Decision Tree

The Decision Tree achieved an accuracy of **73.77%**, precision of **70.00%**, recall of **75.00%**, F1-Score of **72.41%**, and ROC-AUC of **78.90%**.

A maximum tree depth of **5** was used to restrict the complexity of the Decision Tree.

Although Decision Tree provides an intuitive rule-based classification structure, its performance was lower than both SVM and Naive Bayes for the selected dataset and evaluation procedure.

7.10 Overall Model Selection

The final comparison can be summarized as follows:

Evaluation Criterion	Best Algorithm
Accuracy	SVM
Precision	SVM
Recall	SVM
F1-Score	SVM
ROC-AUC	SVM
Overall Best Model	SVM

The selection results identify **SVM as the best model based on Accuracy, F1-Score, and ROC-AUC.**

7.11 Conclusion of Best Model Analysis

Based on the experimental results, **Support Vector Machine with an RBF kernel is the strongest-performing algorithm among the three evaluated methods.** It achieved **88.52% accuracy, 83.87% precision, 92.86% recall, 88.14% F1-Score, and 96.43% ROC-AUC.**

The results demonstrate that SVM provides both strong overall classification accuracy and effective discrimination between the two target classes. Therefore, SVM is selected as the preferred model for the subsequent analysis.

8. CONCLUSION

8.1 Overview

Heart disease is a major health concern, and accurate classification of disease status can support early identification of individuals who may require further medical evaluation. Machine learning provides an effective approach for analyzing patient-related attributes and identifying patterns associated with heart disease.

This study evaluated three supervised machine learning algorithms: **Support Vector Machine (SVM)**, **Gaussian Naive Bayes**, and **Decision Tree** using the Heart Disease Cleveland Dataset. The dataset contains **303 records and 14 attributes**, including demographic, clinical, and diagnostic features. Appropriate preprocessing was performed before model training, including categorical feature encoding, numerical feature scaling, and stratified division of the data into training and testing sets.

8.2 Summary of Results

The three algorithms were evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM	88.52%	83.87%	92.86%	88.14%	96.43%
Naive Bayes	72.13%	64.10%	89.29%	74.63%	82.68%
Decision Tree	73.77%	70.00%	75.00%	72.41%	78.90%

The results demonstrate that **SVM achieved the best overall performance**. It obtained the highest Accuracy, Precision, Recall, F1-Score, and ROC-AUC among the three evaluated algorithms.

8.3 Significance of SVM Performance

The SVM classifier achieved an accuracy of **88.52%**, indicating strong classification capability on the test data. Its recall of **92.86%** demonstrates its ability to identify a high proportion of positive heart-disease cases.

The F1-Score of **88.14%** indicates a strong balance between precision and recall. Furthermore, the ROC-AUC of **96.43%** demonstrates excellent discrimination between the two target classes within the evaluated data.

The RBF kernel used by SVM enables the classifier to model nonlinear relationships among the transformed input features, which contributes to its strong predictive performance.

8.4 Importance of Data Preprocessing

The performance of the classification algorithms was supported by appropriate data preprocessing. Categorical attributes were converted into numerical representations using **One-Hot Encoding**, while numerical attributes were standardized using **StandardScaler**.

The data was then divided into training and testing subsets using a stratified 80:20 split. This preprocessing approach ensures that the algorithms receive appropriately formatted input and that numerical features operate on comparable scales.

8.5 Overall Findings

The comparative analysis establishes several important findings:

1. **SVM was the best-performing algorithm** among the three evaluated methods.
2. SVM achieved **88.52% accuracy**.
3. SVM achieved the highest **92.86% recall**, which is valuable for identifying positive cases.
4. SVM achieved the highest **88.14% F1-Score**.
5. SVM achieved the highest **96.43% ROC-AUC**.
6. Gaussian Naive Bayes provided relatively high recall but lower precision and overall accuracy.
7. Decision Tree provided interpretable classification rules but achieved lower predictive performance than SVM.
8. Proper preprocessing played an important role in preparing the dataset for machine learning classification.

The model-selection analysis also identified SVM as the best algorithm according to Accuracy, F1-Score, and ROC-AUC.

8.6 Final Conclusion

In conclusion, the comparative analysis demonstrates that **Support Vector Machine with an RBF kernel provides the most effective classification performance for the selected Heart Disease Cleveland Dataset**. Its combination of high accuracy, strong recall, balanced F1-Score, and excellent ROC-AUC makes it the preferred algorithm among SVM, Gaussian Naive Bayes, and Decision Tree.

However, the obtained results should be interpreted within the scope of the available dataset and experimental methodology. Further improvements can be achieved through larger datasets, cross-validation, hyperparameter optimization, feature selection, external validation, and explainable machine learning techniques.

Overall, the analysis demonstrates the potential of machine learning-based classification for supporting heart-disease risk identification while highlighting the importance of appropriate preprocessing, algorithm selection, and comprehensive performance evaluation.

REFERENCES

- [1] R. Detrano, A. Janosi, W. Steinbrunn, M. Pfisterer, M. Schmid, J. Sandhu, K. H. Guppy, S. Lee, and V. Froelicher, "International application of a new probability algorithm for the diagnosis of coronary artery disease," *The American Journal of Cardiology*, vol. 64, no. 5, pp. 304–310, 1989.
- [2] V. Vapnik, *The Nature of Statistical Learning Theory*, 2nd ed. New York, NY, USA: Springer, 2000.
- [3] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.
- [4] T. F. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [5] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.
- [6] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [7] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.
- [8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [10] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.
- [11] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [12] J. Han, J. Pei, and M. Kamber, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA, USA: Morgan Kaufmann, 2011.
- [13] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [14] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [15] J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA, USA: Morgan Kaufmann, 1993.